In [1]:

```python
# PROJECT : DineTrack - Restaurant Sales Analytics & Insights.

# Disclaimer: The dataset used in this analysis is simulated (dummy data) and does not contain real customer information.
# It is intended solely for practice and demonstration of data analysis techniques.

# Leverages MySQL to analyze and visualize restaurant sales data, uncovering valuable insights into revenue trends and customer preferences.
# Examining key metrics.
# The analysis aims to identify opportunities for optimizing sales strategies.
# The insights generated will assist restaurant in making data-driven decisions to enhance profitability and customer satisfaction.

!pip install duckdb
!pip install --upgrade pip
import pandas as pd
import duckdb

# load CSV into dataframe

df = pd.read_csv("DineTrack_Restaurant_Sales.csv")
```

```
Requirement already satisfied: duckdb in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (1.4.0)
Requirement already satisfied: pip in C:\Users\sarth\AppData\Local\Programs\Python\Python312\Lib\site-packages (26.1)
```

In [2]:

```python
# 1) TO View structure of table.

res = duckdb.query("DESC df").to_df()
display(res)
```

	column_name	column_type	null	key	default	extra
0	OrderID	VARCHAR	YES	None	None	None
1	OrderDate	VARCHAR	YES	None	None	None
2	OrderDatetimeISO	VARCHAR	YES	None	None	None
3	OrderType	VARCHAR	YES	None	None	None
4	TableNo	DOUBLE	YES	None	None	None
5	TableSection	VARCHAR	YES	None	None	None
6	CustomerID	BIGINT	YES	None	None	None
7	CustomerName	VARCHAR	YES	None	None	None
8	Gender	VARCHAR	YES	None	None	None
9	Phone	BIGINT	YES	None	None	None
10	Email	VARCHAR	YES	None	None	None
11	NumLineItems	BIGINT	YES	None	None	None
12	ItemSummary	VARCHAR	YES	None	None	None
13	Subtotal	DOUBLE	YES	None	None	None
14	ServiceCharge	DOUBLE	YES	None	None	None
15	Discount	DOUBLE	YES	None	None	None
16	Tax	DOUBLE	YES	None	None	None
17	Tip	DOUBLE	YES	None	None	None
18	FinalAmount	DOUBLE	YES	None	None	None
19	OrderStatus	VARCHAR	YES	None	None	None
20	PaymentMode	VARCHAR	YES	None	None	None
21	DeliveryOrPickupTime	VARCHAR	YES	None	None	None
22	StaffID	VARCHAR	YES	None	None	None
23	Notes	VARCHAR	YES	None	None	None
24	OrderYear	BIGINT	YES	None	None	None
25	OrderMonth	BIGINT	YES	None	None	None
26	OrderMonthName	VARCHAR	YES	None	None	None

In [3]:

```python
# 2) To view records (also rows and columns).

res = duckdb.query("SELECT * FROM df").to_df()
display(res)
```

	OrderID	OrderDate	OrderDatetimeISO	OrderType	TableNo	TableSection	CustomerID	CustomerName	Gender	Phone	...	Tip	FinalAmount	OrderStatus	PaymentMode	DeliveryOrPickupTime	StaffID	Notes	OrderYear	OrderMonth	OrderMonthName
0	O0011386	01/06/2024	2024-06-01 06:53:10	Delivery	NaN	None	801	Ekiya Bath	None	913290282270	...	NaN	216.52	Completed	Debit Card	06-01-2024	None	None	2024	6	Jun
1	O0006867	2024-12-28	2024-12-28 11:02:17	Dine-in	28.0	Rooftop	978	Aarnav Kade	Male	1061537221	...	10.0	2103.74	Completed	Debit Card	None	S132	None	2024	12	Dec
2	O0001324	04/06/2024	2024-06-04 18:14:35	Dine-in	1.0	A	232	Adya Virk	Female	915642087375	...	50.0	3754.22	Completed	UPI	None	S109	None	2024	6	Jun
3	O0002253	22/06/2024	2024-06-22 21:35:47	Delivery	NaN	None	1859	Charita Radhakrishnan	Male	9894848170	...	30.0	994.14	Completed	None	22-Jun-2024	None	None	2024	6	Jun
4	O0011278	12/06/2024	2024-06-12 19:05:54	Dine-in	22.0	C	1005	Ansh Kibe	Female	2586314417	...	NaN	1685.25	Completed	Debit Card	None	S102	None	2024	6	Jun
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12595	O0011965	07/08/2024	2024-08-07 04:21:46	Dine-in	5.0	C	1254	Falan Gupta	Female	918233921883	...	50.0	1103.50	Completed	Wallet	None	S147	None	2024	8	Aug
12596	O0005192	02-05-2025	2025-02-05 07:42:50	Dine-in	29.0	Patio	616	Upma Dhingra	None	3590960567	...	NaN	2175.34	Completed	Cash	None	S178	None	2025	2	Feb
12597	O0005391	2025-03-10	2025-03-10 06:34:25	Delivery	NaN	None	1896	Timothy Butala	Male	4890304588	...	NaN	94.37	Completed	Credit Card	2025-03-10	None	None	2025	3	Mar
12598	O0000861	07/07/2024	2024-07-07 15:40:19	Dine-in	4.0	None	2682	Yashawini Sahota	None	3680537380	...	100.0	838.63	Completed	Credit Card	None	S113	None	2024	7	Jul
12599	O0007271	20-Jul-2024	2024-07-20 03:30:50	Delivery	NaN	None	830	Ganga Kaul	Male	8628950905	...	20.0	2591.40	Completed	Debit Card	20/07/2024	None	None	2024	7	Jul

12600 rows × 27 columns

In [4]:

```python
# 3) Total number of records.

res = duckdb.query("SELECT COUNT(*) AS total_rows FROM df").to_df()
res["total_rows"] = res["total_rows"].map("{:,}".format)
display(res)
```

	total_rows
0	12,600

In [5]:

```python
# 4) Find NULL values count in important columns.

res = duckdb.query("""
SELECT
  COUNT(*) FILTER (WHERE CustomerName IS NULL) AS missing_customer_name,
  COUNT(*) FILTER (WHERE PaymentMode IS NULL) AS missing_payment_mode,
  COUNT(*) FILTER (WHERE FinalAmount IS NULL) AS missing_final_amount,
  COUNT(*) FILTER (WHERE OrderDatetimeISO IS NULL) AS missing_order_date
FROM df
""").to_df()
for col in ["missing_customer_name", "missing_payment_mode"]:
    res[col] = res[col].map("{:,}".format)
display(res)
```

	missing_customer_name	missing_payment_mode	missing_final_amount	missing_order_date
0	58	2,300	0	0

In [6]:

```python
# 5) Find duplicate OrderIDs.

res = duckdb.query("""
SELECT OrderID, COUNT(*) AS dup_count
FROM df
GROUP BY OrderID
HAVING COUNT(*) > 1
""").to_df()
display(res)
```

	OrderID	dup_count
0	O0005829	2
1	O0005640	2
2	O0003756	2
3	O0000928	2
4	O0001563	2
...	...	...
595	O0011112	2
596	O0009477	2
597	O0001383	2
598	O0001441	2
599	O0007974	2

600 rows × 2 columns

In [7]:

```python
# 6) To remove Duplicates records.

df = df.drop_duplicates(keep="first")
```

In [8]:

```python
# 7) To view records (also rows and columns) after dupliciates removal.

res = duckdb.query("SELECT * FROM df").to_df()
display(res)
```

	OrderID	OrderDate	OrderDatetimeISO	OrderType	TableNo	TableSection	CustomerID	CustomerName	Gender	Phone	...	Tip	FinalAmount	OrderStatus	PaymentMode	DeliveryOrPickupTime	StaffID	Notes	OrderYear	OrderMonth	OrderMonthName
0	O0011386	01/06/2024	2024-06-01 06:53:10	Delivery	NaN	None	801	Ekiya Bath	None	913290282270	...	NaN	216.52	Completed	Debit Card	06-01-2024	None	None	2024	6	Jun
1	O0006867	2024-12-28	2024-12-28 11:02:17	Dine-in	28.0	Rooftop	978	Aarnav Kade	Male	1061537221	...	10.0	2103.74	Completed	Debit Card	None	S132	None	2024	12	Dec
2	O0001324	04/06/2024	2024-06-04 18:14:35	Dine-in	1.0	A	232	Adya Virk	Female	915642087375	...	50.0	3754.22	Completed	UPI	None	S109	None	2024	6	Jun
3	O0002253	22/06/2024	2024-06-22 21:35:47	Delivery	NaN	None	1859	Charita Radhakrishnan	Male	9894848170	...	30.0	994.14	Completed	None	22-Jun-2024	None	None	2024	6	Jun
4	O0011278	12/06/2024	2024-06-12 19:05:54	Dine-in	22.0	C	1005	Ansh Kibe	Female	2586314417	...	NaN	1685.25	Completed	Debit Card	None	S102	None	2024	6	Jun
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12165	O0011965	07/08/2024	2024-08-07 04:21:46	Dine-in	5.0	C	1254	Falan Gupta	Female	918233921883	...	50.0	1103.50	Completed	Wallet	None	S147	None	2024	8	Aug
12166	O0005192	02-05-2025	2025-02-05 07:42:50	Dine-in	29.0	Patio	616	Upma Dhingra	None	3590960567	...	NaN	2175.34	Completed	Cash	None	S178	None	2025	2	Feb
12167	O0005391	2025-03-10	2025-03-10 06:34:25	Delivery	NaN	None	1896	Timothy Butala	Male	4890304588	...	NaN	94.37	Completed	Credit Card	2025-03-10	None	None	2025	3	Mar
12168	O0000861	07/07/2024	2024-07-07 15:40:19	Dine-in	4.0	None	2682	Yashawini Sahota	None	3680537380	...	100.0	838.63	Completed	Credit Card	None	S113	None	2024	7	Jul
12169	O0007271	20-Jul-2024	2024-07-20 03:30:50	Delivery	NaN	None	830	Ganga Kaul	Male	8628950905	...	20.0	2591.40	Completed	Debit Card	20/07/2024	None	None	2024	7	Jul

12170 rows × 27 columns

In [9]:

```python
# 8) Identify rows with negative or zero amounts (invalid data).

res = duckdb.query("""
SELECT OrderID, Subtotal, Discount, FinalAmount
FROM df
WHERE FinalAmount <= 0
""").to_df()
display(res)
```

OrderID	Subtotal	Discount	FinalAmount

In [10]:

```python
# 9) Standardize PaymentMode values (replace NULL with 'UNKNOWN').

res = duckdb.query("""
SELECT OrderID, COALESCE(PaymentMode, 'UNKNOWN') AS PaymentMode
FROM df
""").to_df()
display(res)
```

	OrderID	PaymentMode
0	O0011386	Debit Card
1	O0006867	Debit Card
2	O0001324	UPI
3	O0002253	UNKNOWN
4	O0011278	Debit Card
...	...	...
12165	O0011965	Wallet
12166	O0005192	Cash
12167	O0005391	Credit Card
12168	O0000861	Credit Card
12169	O0007271	Debit Card

12170 rows × 2 columns

In [11]:

```python
# 10) Clean CustomerName (TRIM).

res = duckdb.query("""
SELECT OrderID, TRIM(CustomerName) AS CleanName
FROM df
""").to_df()
display(res)
```

	OrderID	CleanName
0	O0011386	Ekiya Bath
1	O0006867	Aarnav Kade
2	O0001324	Adya Virk
3	O0002253	Charita Radhakrishnan
4	O0011278	Ansh Kibe
...	...	...
12165	O0011965	Falan Gupta
12166	O0005192	Upma Dhingra
12167	O0005391	Timothy Butala
12168	O0000861	Yashawini Sahota
12169	O0007271	Ganga Kaul

12170 rows × 2 columns

In [12]:

```python
# 11) Total revenue & orders.

res = duckdb.query("""
SELECT COUNT(*) AS total_orders,
       SUM(FinalAmount) AS total_revenue
FROM df
""").to_df()
display(res)
```

	total_orders	total_revenue
0	12170	25430811.17

In [13]:

```python
# 12) Unique customers.

res = duckdb.query("SELECT COUNT(DISTINCT CustomerID) AS unique_customers FROM df").to_df()
res["unique_customers"] = res["unique_customers"].map("{:,}".format)
display(res)
```

	unique_customers
0	2,949

In [14]:

```python
# 13) Orders by order type.

res = duckdb.query("""
SELECT OrderType, COUNT(*) AS total_orders
FROM df
GROUP BY OrderType
""").to_df()
res["total_orders"] = res["total_orders"].map("{:,}".format)
display(res)
```

	OrderType	total_orders
0	Dine-in	6,632
1	Takeaway	2,450
2	Delivery	3,088

In [15]:

```python
# 14) Orders by payment mode.

res = duckdb.query("""
SELECT PaymentMode, COUNT(*) AS total_orders
FROM df
GROUP BY PaymentMode
ORDER BY total_orders DESC
""").to_df()
res["total_orders"] = res["total_orders"].map("{:,}".format)
display(res)
```

	PaymentMode	total_orders
0	None	2,224
1	Debit Card	2,035
2	Cash	2,008
3	UPI	1,971
4	Wallet	1,968
5	Credit Card	1,964

In [16]:

```python
# 15) Orders by order status.

res = duckdb.query("""
SELECT OrderStatus,COUNT(*) AS order_count
FROM df
GROUP BY OrderStatus
""").to_df()
res["order_count"] = res["order_count"].map("{:,}".format)
display(res)
```

	OrderStatus	order_count
0	Pending	572
1	Completed	11,037
2	Refunded	223
3	Cancelled	338

In [17]:

```python
# 16) Orders by order status (Year - Month ).

res = duckdb.query("""
SELECT
  strftime('%Y', CAST(OrderDatetimeISO AS TIMESTAMP)) AS order_year,
  strftime('%Y-%m', CAST(OrderDatetimeISO AS TIMESTAMP)) AS order_month,
  OrderStatus,
  COUNT(*) AS order_count
FROM df
GROUP BY order_year, order_month, OrderStatus
ORDER BY order_year, order_month, order_count DESC
""").to_df()
res["order_count"] = res["order_count"].map("{:,}".format)
display(res)
```

	order_year	order_month	OrderStatus	order_count
0	2024	2024-06	Completed	876
1	2024	2024-06	Pending	39
2	2024	2024-06	Cancelled	28
3	2024	2024-06	Refunded	21
4	2024	2024-07	Completed	870
5	2024	2024-07	Pending	47
6	2024	2024-07	Cancelled	31
7	2024	2024-07	Refunded	19
8	2024	2024-08	Completed	864
9	2024	2024-08	Pending	50
10	2024	2024-08	Cancelled	30
11	2024	2024-08	Refunded	11
12	2024	2024-09	Completed	916
13	2024	2024-09	Pending	39
14	2024	2024-09	Cancelled	23
15	2024	2024-09	Refunded	20
16	2024	2024-10	Completed	837
17	2024	2024-10	Pending	48
18	2024	2024-10	Cancelled	29
19	2024	2024-10	Refunded	20
20	2024	2024-11	Completed	811
21	2024	2024-11	Pending	42
22	2024	2024-11	Cancelled	23
23	2024	2024-11	Refunded	13
24	2024	2024-12	Completed	854
25	2024	2024-12	Pending	45
26	2024	2024-12	Cancelled	16
27	2024	2024-12	Refunded	15
28	2025	2025-01	Completed	836
29	2025	2025-01	Pending	39
30	2025	2025-01	Refunded	24
31	2025	2025-01	Cancelled	14
32	2025	2025-02	Completed	768
33	2025	2025-02	Pending	42
34	2025	2025-02	Cancelled	26
35	2025	2025-02	Refunded	13
36	2025	2025-03	Completed	842
37	2025	2025-03	Pending	44
38	2025	2025-03	Cancelled	28
39	2025	2025-03	Refunded	16
40	2025	2025-04	Completed	839
41	2025	2025-04	Pending	48
42	2025	2025-04	Cancelled	31
43	2025	2025-04	Refunded	14
44	2025	2025-05	Completed	907
45	2025	2025-05	Pending	44
46	2025	2025-05	Cancelled	34
47	2025	2025-05	Refunded	20
48	2025	2025-06	Completed	817
49	2025	2025-06	Pending	45
50	2025	2025-06	Cancelled	25
51	2025	2025-06	Refunded	17

In [18]:

```python
# 17) Orders by order status (Yearly).

res = duckdb.query("""
SELECT
  strftime('%Y', CAST(OrderDatetimeISO AS TIMESTAMP)) AS order_year,
  OrderStatus,
  COUNT(*) AS order_count
FROM df
GROUP BY order_year, OrderStatus
ORDER BY order_year, order_count DESC
""").to_df()
res["order_count"] = res["order_count"].map("{:,}".format)
display(res)
```

	order_year	OrderStatus	order_count
0	2024	Completed	6,028
1	2024	Pending	310
2	2024	Cancelled	180
3	2024	Refunded	119
4	2025	Completed	5,009
5	2025	Pending	262
6	2025	Cancelled	158
7	2025	Refunded	104

In [19]:

```python
# 18) Monthly revenue trend.

res = duckdb.query("""
SELECT strftime('%Y-%m', CAST(OrderDatetimeISO AS TIMESTAMP)) AS order_month,
       SUM(FinalAmount) AS revenue
FROM df
GROUP BY order_month
ORDER BY order_month
""").to_df()
res["revenue"] = res["revenue"].map("{:,.2f}".format)
display(res)
```

	order_month	revenue
0	2024-06	2,092,537.93
1	2024-07	2,015,361.63
2	2024-08	2,001,242.29
3	2024-09	2,055,508.82
4	2024-10	2,004,995.48
5	2024-11	1,843,915.22
6	2024-12	1,885,538.09
7	2025-01	1,989,113.20
8	2025-02	1,775,266.36
9	2025-03	1,932,043.11
10	2025-04	1,939,489.57
11	2025-05	2,033,210.71
12	2025-06	1,862,588.76

In [20]:

```python
# 19) Yearly revenue trend.

res = duckdb.query("""
SELECT
    strftime('%Y', CAST(OrderDatetimeISO AS TIMESTAMP)) AS order_year,
    SUM(FinalAmount) AS revenue
FROM df
GROUP BY order_year
ORDER BY order_year
""").to_df()
res["revenue"] = res["revenue"].map("{:,.2f}".format)
display(res)
```

	order_year	revenue
0	2024	13,899,099.46
1	2025	11,531,711.71

In [21]:

```python
# 20) Top 5 customers by spend.

res = duckdb.query("""
SELECT
    OrderYear,
    CustomerID,
    CustomerName,
    COUNT(*) AS total_orders,
    SUM(FinalAmount) AS total_spent
FROM df
GROUP BY CustomerID, CustomerName, OrderYear
ORDER BY SUM(FinalAmount) DESC
LIMIT 5
""").to_df()
res["total_spent"] = res["total_spent"].map("{:,.2f}".format)
display(res)
```

	OrderYear	CustomerID	CustomerName	total_orders	total_spent
0	2024	354	Yashvi Raval	4	21,858.81
1	2024	34	Lajita Sood	4	21,113.17
2	2025	72	Yochana Chopra	7	20,866.02
3	2024	2094	Ishwar Ramanathan	6	20,775.29
4	2024	626	Atharv Dube	6	20,603.29

In [22]:

```python
# 21) Orders & revenue by weekday.

res = duckdb.query("""
SELECT
    strftime('%A', CAST(OrderDatetimeISO AS TIMESTAMP)) AS weekday,
    COUNT(*) AS orders,
    SUM(FinalAmount) AS revenue
FROM df
GROUP BY weekday
ORDER BY COUNT(*) DESC
""").to_df()
res["revenue"] = res["revenue"].map("{:,.2f}".format)
display(res)
```

	weekday	orders	revenue
0	Sunday	1812	3,769,471.45
1	Saturday	1808	3,714,767.68
2	Friday	1763	3,699,834.37
3	Monday	1736	3,726,950.91
4	Tuesday	1700	3,625,027.55
5	Wednesday	1695	3,446,960.72
6	Thursday	1656	3,447,798.49

In [23]:

```python
# 22) Orders by hour of day.

res = duckdb.query("""
SELECT
    CAST(strftime('%H', CAST(OrderDatetimeISO AS TIMESTAMP)) AS INTEGER) AS order_hour,
    strftime('%p', CAST(OrderDatetimeISO AS TIMESTAMP)) AS am_pm,
    COUNT(*) AS total_orders
FROM df
GROUP BY order_hour, am_pm
ORDER BY order_hour
""").to_df()
display(res)
```

	order_hour	am_pm	total_orders
0	0	AM	512
1	1	AM	523
2	2	AM	482
3	3	AM	490
4	4	AM	511
5	5	AM	479
6	6	AM	505
7	7	AM	492
8	8	AM	532
9	9	AM	502
10	10	AM	481
11	11	AM	529
12	12	PM	508
13	13	PM	516
14	14	PM	526
15	15	PM	517
16	16	PM	501
17	17	PM	497
18	18	PM	469
19	19	PM	520
20	20	PM	495
21	21	PM	514
22	22	PM	551
23	23	PM	518

In [24]:

```python
# 23) Top 10 items by quantity.

res = duckdb.query("""
SELECT
  item_name,
  SUM(quantity) AS total_quantity,
  COUNT(*) AS times_ordered
FROM (
  SELECT
    TRIM(REGEXP_REPLACE(raw_item, 'x[0-9]+', '')) AS item_name,
    CAST(REGEXP_EXTRACT(raw_item, 'x([0-9]+)', 1) AS INTEGER) AS quantity
  FROM (
    SELECT TRIM(item) AS raw_item
    FROM df,
    UNNEST(STRING_SPLIT(ItemSummary, ';')) AS t(item)
  )
  WHERE raw_item <> ''
)
GROUP BY item_name
ORDER BY SUM(quantity) DESC
LIMIT 10
""").to_df()
res["total_quantity"] = res["total_quantity"].map("{:,.2f}".format)
res["times_ordered"] = res["times_ordered"].map("{:,.2f}".format)
display(res)
```

	item_name	total_quantity	times_ordered
0	Popcorn	4,111.00	2,048.00
1	Lunch Combo	4,064.00	2,007.00
2	Family Platter	4,034.00	2,033.00
3	Chicken Wings	4,008.00	2,005.00
4	Garlic Bread	3,989.00	1,982.00
5	Breakfast Combo	3,957.00	1,980.00
6	Cheesecake	3,123.00	1,535.00
7	Brownie	3,080.00	1,540.00
8	Paneer Tikka	3,072.00	1,528.00
9	Gulab Jamun	3,057.00	1,524.00

In [25]:

```python
# 24) Customer frequency buckets.

res = duckdb.query("""
SELECT customer_type, COUNT(*) AS num_customers
FROM (
  SELECT CustomerID,
         CASE WHEN order_count = 1 THEN 'One-time'
              WHEN order_count BETWEEN 2 AND 5 THEN 'Repeat (2-5)'
              ELSE 'Loyal (6+)' END AS customer_type
  FROM (
    SELECT CustomerID, COUNT(*) AS order_count
    FROM df
    GROUP BY CustomerID
  )
)
GROUP BY customer_type
""").to_df()
res["num_customers"] = res["num_customers"].map("{:,.2f}".format)
display(res)
```

	customer_type	num_customers
0	Repeat (2-5)	2,102.00
1	Loyal (6+)	654.00
2	One-time	193.00